

Stock Price Prediction and Trend Classification

A Complete Machine Learning Project Documentation

Machine Learning Project Report

September 9, 2025

Abstract

This project report presents a comprehensive machine learning approach to stock price prediction using Netflix (NFLX) historical data. We implement both regression and classification models to predict next-day closing price returns and trend directions. The project addresses key challenges including data leakage prevention, feature engineering with technical indicators, multicollinearity treatment, and proper evaluation metrics for financial time series. Through extensive experimentation with linear models, tree-based algorithms, and ensemble methods, we demonstrate that simple linear approaches outperform complex models for this task, achieving RMSE values of approximately 2.5% while highlighting the inherent difficulty of stock market prediction.

Contents

1	Introduction	4
1.1	Project Overview	4
1.2	Technology Stack	4
1.3	Dataset Description	4
2	Problem Definition	4
2.1	Mathematical Formulation	4
2.1.1	Regression Problem	4
2.1.2	Classification Problem	4
3	Data Collection and Sources	5
3.1	Primary Data Source	5
4	Key Challenges and Solutions	5
4.1	Target Variable Creation	5
4.2	Temporal Feature Encoding	5
4.3	Data Leakage Prevention	5
4.4	Non-Stationarity Treatment	6
4.5	Multicollinearity Treatment	6

5	Feature Engineering	6
5.1	Technical Indicators Implementation	6
5.1.1	Price-based Features	6
5.1.2	Volatility Measures	7
5.1.3	Volume Indicators	7
5.1.4	Momentum Indicators	7
5.1.5	Trend Indicators	7
5.1.6	Volatility Bands	8
5.2	Final Feature Set	8
6	Data Preprocessing	8
6.1	Missing Values Treatment	8
6.2	Feature Scaling	9
6.3	Feature Selection Methods	9
6.3.1	Recursive Feature Elimination (RFE)	9
6.3.2	Lasso Regularization	9
7	Model Implementation	10
7.1	Regression Model Categories	10
7.1.1	Linear Models	10
7.1.2	Tree-based Models	10
7.1.3	Instance-based and Kernel Models	10
7.2	Regression Hyperparameter Tuning	10
7.3	Classification Model Implementation	11
7.3.1	Data Setup for Classification	11
7.3.2	Logistic Regression Classification	11
7.3.3	Decision Tree Classification	12
7.3.4	SVM Classification	12
7.3.5	KNN Classification	12
7.4	Classification Hyperparameter Tuning	13
7.4.1	Logistic Regression Grid Search	13
7.4.2	Decision Tree Grid Search	13
7.4.3	SVM Grid Search	13
7.4.4	KNN Grid Search	14
8	Results Analysis	14
8.1	Regression Model Performance	14
8.1.1	Initial Model Results	14
8.1.2	Hyperparameter Tuned Regression Results	14
8.1.3	Tree-based and Complex Models	15
8.2	Classification Model Performance	15
8.2.1	Logistic Regression Results	15
8.2.2	Decision Tree Classification Results	15
8.2.3	SVM Classification Results	15
8.2.4	KNN Classification Results	15
8.3	Key Observations and Insights	16
8.3.1	Regression Model Analysis	16
8.3.2	Classification Model Analysis	16
8.3.3	Cross-Problem Insights	16

8.3.4	Model Behavior Patterns	17
9	Lessons Learned	17
9.1	Successful Approaches	17
9.1.1	Regression Models	17
9.1.2	Classification Models	17
9.2	Unsuccessful Approaches	17
9.2.1	Regression Models	17
9.2.2	Classification Models	18
9.3	Critical Insights	18
9.3.1	Financial Market Reality	18
9.3.2	Methodological Lessons	18
9.4	Model Selection Guidelines	18
9.4.1	For Regression Tasks	18
9.4.2	For Classification Tasks	18
10	Future Improvements	19
10.1	Advanced Models	19
10.2	Feature Enhancement	19
10.3	Ensemble Approaches	19
10.4	Alternative Targets	19
11	Conclusion	19
11.1	Performance Summary	20
11.1.1	Regression Results	20
11.1.2	Classification Results	20
11.2	Key Methodological Contributions	20
11.2.1	Technical Achievements	20
11.2.2	Evaluation Framework	20
11.3	Practical Implications	21
11.3.1	For Practitioners	21
11.3.2	For Researchers	21
11.4	Fundamental Insights	21
11.4.1	Market Efficiency Validated	21
11.4.2	Methodology Matters More Than Algorithms	21
11.4.3	Domain Knowledge Essential	22
11.5	Future Research Directions	22
11.6	Final Assessment	22
12	References	22

1 Introduction

1.1 Project Overview

The objective of this project is to develop a machine learning system capable of predicting stock price movements using two distinct approaches: regression for predicting next-day closing price returns, and classification for predicting trend direction (up/down movements).

1.2 Technology Stack

The implementation utilizes a comprehensive technology stack:

- **Data Collection:** yfinance, pandas
- **Feature Engineering:** numpy
- **Machine Learning:** scikit-learn (Linear Regression, Ridge, Lasso, Logistic Regression, Decision Tree, KNN, SVM)
- **Visualization:** matplotlib, seaborn
- **Analysis:** statsmodels (VIF analysis)

1.3 Dataset Description

The dataset consists of Netflix (NFLX) stock data sourced from YFinance, containing historical daily OHLCV (Open, High, Low, Close, Volume) data enhanced with over 20 engineered technical indicators.

2 Problem Definition

2.1 Mathematical Formulation

The project addresses two primary prediction tasks:

2.1.1 Regression Problem

Predict the next-day percentage return:

$$\text{Target_Return_next} = \frac{\text{Close}_{\text{tomorrow}}}{\text{Close}_{\text{today}}} - 1 \quad (1)$$

2.1.2 Classification Problem

Predict the next-day direction:

$$\text{Target_Dir_next} = \begin{cases} 1 & \text{if } \text{Close}_{\text{tomorrow}} > \text{Close}_{\text{today}} \\ 0 & \text{otherwise} \end{cases} \quad (2)$$

3 Data Collection and Sources

3.1 Primary Data Source

Data collection was performed using the yfinance library to fetch Netflix (NFLX) historical stock data directly from Yahoo Finance. The yfinance API provides reliable access to OHLCV data with proper handling of stock splits and dividends.

The raw data structure follows the standard OHLCV format:

```
1 columns = ['Date', 'Open', 'High', 'Low', 'Close', 'Volume']
```

Listing 1: Basic OHLCV Structure

4 Key Challenges and Solutions

4.1 Target Variable Creation

Challenge: No existing classification target for trend prediction.

Solution: Created binary classification target through iterative comparison:

```
1 df['target'] = np.zeros(len(df), dtype=int)
2 for i in range(len(df['Close'])-1):
3     if df['Close'].iloc[i] >= df['Close'].iloc[i+1]:
4         df.loc[df.index[i], 'target'] = 1 # Down/Flat
5     else:
6         df.loc[df.index[i], 'target'] = 0 # Up
```

Listing 2: Target Variable Creation

4.2 Temporal Feature Encoding

Challenge: Proper encoding of date-based features without introducing bias.

Solution: Extraction of cyclical temporal components:

```
1 df['DayOfWeek'] = df['Date'].dt.dayofweek # 0=Monday
2 df['Month'] = df['Date'].dt.month
3 df['Year'] = df['Date'].dt.year
```

Listing 3: Temporal Feature Extraction

4.3 Data Leakage Prevention

Challenge: Avoiding look-ahead bias in feature construction.

Solution: Implementation of lagged and shifted rolling features:

```
1 # Lagged features
2 df['Close_Lag1'] = df['Close'].shift(1)
3 df['Close_Lag2'] = df['Close'].shift(2)
4
5 # Rolling features with shift to avoid leakage
```

```

6 df['Volatility_10d'] = df['Close_Return'].rolling(window=10).std()
  .shift(1)
7 df['Volume_Mean20'] = df['Volume'].rolling(20).mean().shift(1)

```

Listing 4: Lagged Feature Implementation

4.4 Non-Stationarity Treatment

Challenge: Raw stock prices exhibit non-stationary behavior.

Solution: Transformation to percentage returns:

```

1 # Percentage returns (more stationary)
2 df['Close_Return'] = df['Close'].pct_change()
3 df['Close_return_per'] = df['Close_Return'] * 100
4
5 # Log returns (alternative approach)
6 df['LogReturn'] = np.log(df['Close'] / df['Close'].shift(1))

```

Listing 5: Stationarity Enhancement

4.5 Multicollinearity Treatment

Challenge: 28 features exhibited high correlation (VIF > 10).

Solution: Variance Inflation Factor (VIF) analysis and systematic feature removal:

```

1 from statsmodels.stats.outliers_influence import
  variance_inflation_factor
2
3 vif_data = pd.DataFrame()
4 vif_data['Feature'] = X.columns
5 vif_data['VIF'] = [variance_inflation_factor(X.values, i)
6                   for i in range(X.shape[1])]
7
8 # Removed features with VIF > 10
9 redundant_features = ['Open', 'High', 'Low', 'Close', '
10                      Close_return_per',
11                      'LogReturn', 'Close_Lag2', 'MACD_Signal']

```

Listing 6: VIF Analysis Implementation

5 Feature Engineering

5.1 Technical Indicators Implementation

5.1.1 Price-based Features

```

1 # Lagged prices
2 df['Close_Lag1'] = df['Close'].shift(1)
3 df['Close_Lag2'] = df['Close'].shift(2)
4

```

```

5 # Returns
6 df['Close_Return'] = df['Close'].pct_change()
7 df['Open_Return'] = df['Open'].pct_change()
8 df['High_Return'] = df['High'].pct_change()
9 df['Low_Return'] = df['Low'].pct_change()

```

Listing 7: Price-based Feature Engineering

5.1.2 Volatility Measures

Rolling volatility calculation with proper lag implementation:

```

1 # Rolling volatility (10-day)
2 df['Volatility_10d'] = df['Close_Return'].rolling(window=10).std()
   .shift(1)

```

Listing 8: Volatility Feature

5.1.3 Volume Indicators

```

1 # Volume ratios
2 df['Volume_Return'] = df['Volume'].pct_change()
3 df['Volume_Mean20'] = df['Volume'].rolling(20).mean().shift(1)
4 df['Volume_Ratio_20d'] = df['Volume'] / df['Volume_Mean20']

```

Listing 9: Volume-based Features

5.1.4 Momentum Indicators

Relative Strength Index (RSI) implementation:

```

1 def compute_RSI(series, length=14):
2     delta = series.diff()
3     gain = delta.clip(lower=0)
4     loss = -delta.clip(upper=0)
5     avg_gain = gain.ewm(alpha=1/length, adjust=False).mean()
6     avg_loss = loss.ewm(alpha=1/length, adjust=False).mean()
7     rs = avg_gain / (avg_loss + 1e-12)
8     rsi = 100 - (100 / (1 + rs))
9     return rsi
10
11 df['RSI_14'] = compute_RSI(df['Close']).shift(1)

```

Listing 10: RSI Implementation

5.1.5 Trend Indicators

Moving Average Convergence Divergence (MACD):

```

1 # MACD calculation
2 ema12 = df['Close'].ewm(span=12, adjust=False).mean()
3 ema26 = df['Close'].ewm(span=26, adjust=False).mean()

```

```

4 df['MACD'] = (ema12 - ema26).shift(1)
5 df['MACD_Signal'] = df['MACD'].ewm(span=9, adjust=False).mean().
  shift(1)

```

Listing 11: MACD Implementation

5.1.6 Volatility Bands

Bollinger Bands Width calculation:

```

1 # Bollinger Bands Width
2 rolling_mean_20 = df['Close'].rolling(20).mean().shift(1)
3 rolling_std_20 = df['Close'].rolling(20).std().shift(1)
4 bb_upper = rolling_mean_20 + 2 * rolling_std_20
5 bb_lower = rolling_mean_20 - 2 * rolling_std_20
6 df['Bollinger_Band_Width'] = (bb_upper - bb_lower) /
  rolling_mean_20

```

Listing 12: Bollinger Bands Implementation

5.2 Final Feature Set

After VIF analysis and multicollinearity treatment, the final feature set includes:

- Close_Return
- Open_Return, High_Return, Low_Return
- Volume_Return, Volume_Ratio_20d
- Close_Lag1
- Volatility_10d
- RSI_14
- MACD
- Bollinger_Band_Width

6 Data Preprocessing

6.1 Missing Values Treatment

```

1 # Drop NaN values caused by shifting and rolling operations
2 df = df.dropna()

```

Listing 13: Missing Values Handling

6.2 Feature Scaling

Proper scaling implementation to prevent data leakage:

```
1 from sklearn.preprocessing import StandardScaler
2
3 # Split data first to prevent data leakage
4 X_train, X_test, y_train, y_test = train_test_split(X, y,
5     test_size=0.2, random_state=42)
6
7 # Fit scaler on training data only
8 scaler = StandardScaler()
9 X_train_scaled = pd.DataFrame(
10     scaler.fit_transform(X_train),
11     columns=X.columns,
12     index=X_train.index
13 )
14 X_test_scaled = pd.DataFrame(
15     scaler.transform(X_test),
16     columns=X.columns,
17     index=X_test.index
18 )
```

Listing 14: Feature Scaling Implementation

6.3 Feature Selection Methods

6.3.1 Recursive Feature Elimination (RFE)

```
1 from sklearn.feature_selection import RFE
2 from sklearn.linear_model import LinearRegression
3
4 linear_model = LinearRegression()
5 rfe = RFE(linear_model, step=1, n_features_to_select=10)
6 rfe.fit(X_train, y_train)
7 selected_columns = X.columns[rfe.support_]
8
```

Listing 15: RFE Implementation

6.3.2 Lasso Regularization

```
1 from sklearn.linear_model import LassoCV
2
3 lasso = LassoCV(max_iter=100000, random_state=42)
4 lasso.fit(X_train, y_train)
5 selected_features = X.columns[(lasso.coef_ != 0)]
6
```

Listing 16: Lasso Feature Selection

7 Model Implementation

7.1 Regression Model Categories

7.1.1 Linear Models

```
1 regression_linear_pipeline = {
2     'LinearRegression': Pipeline(steps=[('model', LinearRegression
3         ())),
4     'Lasso': Pipeline(steps=[('model', Lasso())]),
5     'Ridge': Pipeline(steps=[('model', Ridge())])
6 }
```

Listing 17: Linear Models Pipeline

7.1.2 Tree-based Models

```
1 regression_tree_pipeline = {
2     'DecisionTreeRegressor': Pipeline(steps=[('model',
3         DecisionTreeRegressor())])
4 }
```

Listing 18: Tree-based Models

7.1.3 Instance-based and Kernel Models

```
1 regression_knn_svm_pipeline = {
2     'KNN': Pipeline(steps=[('scaler', StandardScaler()),
3         ('model', KNeighborsRegressor())]),
4     'SVM': Pipeline(steps=[('scaler', StandardScaler()),
5         ('model', SVR())])
6 }
```

Listing 19: KNN and SVM Models

7.2 Regression Hyperparameter Tuning

```
1 # Ridge Regression parameters
2 param_grid_ridge_lasso = {
3     'model__alpha': [0.01, 0.1, 1.0, 10.0, 50.0, 100.0],
4     'model__max_iter': [100, 1000, 2000, 5000, 10000],
5     'model__random_state': [22, 42, 55]
6 }
7
8 # Grid search with cross-validation
9 grids_linear_regression = {
10     'Ridge': GridSearchCV(regression_linear_pipeline['Ridge'],
11         param_grid_ridge_lasso, cv=5, scoring='r2
12     )
13 }
```

12

}

Listing 20: Regression Hyperparameter Grid Search

7.3 Classification Model Implementation

7.3.1 Data Setup for Classification

```

1 # Classification target and features
2 X_classi = data.drop(['Target_Dir_next', 'Date', 'Target_Return_next',
3                       ', 'target'], axis=1)
4 y_classi = data['Target_Dir_next']
5
6 # Train-test split
7 x_train_classi, x_test_classi, y_train_classi, y_test_classi = \
8     train_test_split(X_classi, y_classi, test_size=0.2,
9                       random_state=42)
10
11 # Feature scaling
12 sc = StandardScaler()
13 x_train_classi = pd.DataFrame(sc.fit_transform(x_train_classi),
14                               columns=X_classi.columns,
15                               index=x_train_classi.index)
16 x_test_classi = pd.DataFrame(sc.transform(x_test_classi),
17                              columns=X_classi.columns,
18                              index=x_test_classi.index)

```

Listing 21: Classification Data Preparation

7.3.2 Logistic Regression Classification

```

1 # Raw Logistic Regression
2 ll = LogisticRegression()
3 ll.fit(x_train_classi, y_train_classi)
4 y_pred_logistic = ll.predict(x_test_classi)
5
6 # Evaluation
7 print("accuracy: ", accuracy_score(y_test_classi, y_pred_logistic))
8 print("confusion matrix: ", confusion_matrix(y_test_classi,
9                                                y_pred_logistic))
10 print("Precision", precision_score(y_test_classi, y_pred_logistic))
11 print("recall: ", recall_score(y_test_classi, y_pred_logistic))
12 print("f1_score: ", f1_score(y_test_classi, y_pred_logistic))

```

Listing 22: Logistic Regression Implementation

7.3.3 Decision Tree Classification

```
1 # Raw Decision Tree
2 dc = DecisionTreeClassifier()
3 dc.fit(x_train_classi, y_train_classi)
4 y_pred_dc = dc.predict(x_test_classi)
5
6 # Evaluation metrics
7 print("accuracy: ", accuracy_score(y_test_classi, y_pred_dc))
8 print("confusion_matrix: ", confusion_matrix(y_test_classi,
9 y_pred_dc))
10 print("Precision", precision_score(y_test_classi, y_pred_dc))
11 print("recall: ", recall_score(y_test_classi, y_pred_dc))
12 print("f1_score: ", f1_score(y_test_classi, y_pred_dc))
```

Listing 23: Decision Tree Classification

7.3.4 SVM Classification

```
1 # Raw SVM
2 svm = SVC()
3 svm.fit(x_train_classi, y_train_classi)
4 y_pred_svm = svm.predict(x_test_classi)
5
6 # Evaluation
7 print("accuracy:", accuracy_score(y_test_classi, y_pred_svm))
8 print("confusion_matrix:", confusion_matrix(y_test_classi,
9 y_pred_svm))
10 print("Precision", precision_score(y_test_classi, y_pred_svm))
11 print("recall:", recall_score(y_test_classi, y_pred_svm))
12 print("f1_score:", f1_score(y_test_classi, y_pred_svm))
```

Listing 24: SVM Classification

7.3.5 KNN Classification

```
1 # Raw KNN
2 knn = KNeighborsClassifier()
3 knn.fit(x_train_classi, y_train_classi)
4 y_pred_knn = knn.predict(x_test_classi)
5
6 # Evaluation
7 print("accuracy: ", accuracy_score(y_test_classi, y_pred_knn))
8 print("confusion_matrix: ", confusion_matrix(y_test_classi,
9 y_pred_knn))
10 print("Precision", precision_score(y_test_classi, y_pred_knn))
11 print("recall: ", recall_score(y_test_classi, y_pred_knn))
12 print("f1_score: ", f1_score(y_test_classi, y_pred_knn))
```

Listing 25: KNN Classification

7.4 Classification Hyperparameter Tuning

7.4.1 Logistic Regression Grid Search

```
1 param_grid_logistic = {
2     'penalty': ['l1', 'l2'],
3     'C': [0.01, 0.1, 1.0, 10.0],
4     'max_iter': [10, 100, 1000, 2000, 10000],
5     'solver': ['liblinear', 'saga', 'lbfgs'],
6     'fit_intercept': [True, False]
7 }
8
9 logistic_grid = GridSearchCV(LogisticRegression(),
10     param_grid_logistic,
11                               cv=5, scoring='accuracy')
12 logistic_grid.fit(x_train_classi, y_train_classi)
13
14 print("Best Params:", logistic_grid.best_params_)
15 print("Best CV Score:", logistic_grid.best_score_)
16 print("Test accuracy:", logistic_grid.score(x_test_classi,
17     y_test_classi))
```

Listing 26: Logistic Regression Hyperparameter Tuning

7.4.2 Decision Tree Grid Search

```
1 param_grid_decision = {
2     'criterion': ['gini', 'entropy'],
3     'max_depth': [None, 1, 10, 20, 50],
4     'min_samples_split': [2, 3, 4, 5, 6, 10],
5     'min_samples_leaf': [1, 2, 3, 4],
6     'max_leaf_nodes': [None, 1, 2, 3]
7 }
8
9 decision_tree_grid = GridSearchCV(DecisionTreeClassifier(),
10     param_grid_decision,
11                               cv=5, scoring='f1')
12 decision_tree_grid.fit(x_train_classi, y_train_classi)
```

Listing 27: Decision Tree Hyperparameter Tuning

7.4.3 SVM Grid Search

```
1 param_grid_svm = {
2     'C': [0.01, 1, 10],
3     'kernel': ['rbf', 'linear', 'poly'],
4     'degree': [2, 3, 4],
5     'probability': [True, False],
6     'gamma': ['scale', 'auto']
7 }
```

```

8 svm_grid = GridSearchCV(SVC(), param_grid_svm, cv=5, scoring='f1')
9 svm_grid.fit(x_train_classi, y_train_classi)
10

```

Listing 28: SVM Hyperparameter Tuning

7.4.4 KNN Grid Search

```

1 param_grid_knn = {
2     'n_neighbors': [2, 4, 3, 5, 6],
3     'weights': ['uniform', 'distance'],
4     'metric': ['euclidean', 'manhattan', 'minkowski']
5 }
6
7 knn_grid = GridSearchCV(KNeighborsClassifier(), param_grid_knn,
8                         cv=5, scoring='f1')
9 knn_grid.fit(x_train_classi, y_train_classi)

```

Listing 29: KNN Hyperparameter Tuning

8 Results Analysis

8.1 Regression Model Performance

8.1.1 Initial Model Results

Table 1: Initial Regression Model Performance

Model	RMSE	R ²	Status
LinearRegression	0.02576	0.0062	Best R ²
Ridge	0.02576	0.0063	Slight improvement
Lasso	0.02590	-0.0046	Negative R ²

8.1.2 Hyperparameter Tuned Regression Results

Table 2: Tuned Linear Regression Models

Model	Best CV R ²	Test R ²	Best Parameters	Performance
LinearRegression	-0.0532	0.0008	fit_intercept: False	Minimal improvement
Ridge	-0.0467	0.0085	=100.0, iter=100	Best performer
Lasso	-0.0203	-0.0046	=0.01, iter=100	Poor performance

Table 3: Complex Regression Model Results

Model	RMSE	Test R ²	Status
DecisionTree (Raw)	0.02576	-1.300	Severe overfitting
DecisionTree (Tuned)	N/A	-0.0665	Still negative
KNN (Tuned)	0.02652	-0.0533	Poor performance
SVR (Tuned)	0.02744	-0.1278	Worst performance

8.1.3 Tree-based and Complex Models

8.2 Classification Model Performance

8.2.1 Logistic Regression Results

Table 4: Logistic Regression Classification Results

Configuration	Accuracy	Precision	Recall	F1-Score	Features	Status
All Features	0.449	0.500	0.529	0.514	28	Baseline
RFE Selected	0.470	0.516	0.581	0.547	10	Improved
Hypertuned	0.514	0.543	0.750	0.630	10	Best

8.2.2 Decision Tree Classification Results

Table 5: Decision Tree Classification Results

Configuration	Accuracy	Precision	Recall	F1-Score	Features	Behavior
All Features	0.543	0.586	0.574	0.580	28	Balanced
RFE Selected	0.482	0.531	0.500	0.515	10	Decreased
Hypertuned	0.555	0.553	0.993	0.711	28	High recall

8.2.3 SVM Classification Results

Table 6: SVM Classification Results

Configuration	Accuracy	Precision	Recall	F1-Score	Features	Notes
All Features	0.567	0.588	0.713	0.645	28	Balanced
RFE Selected	0.494	0.536	0.603	0.567	10	Decreased
Hypertuned	0.547	0.549	0.993	0.707	10	High recall

8.2.4 KNN Classification Results

Table 7: KNN Classification Results

Configuration	Accuracy	Precision	Recall	F1-Score	Features	Performance
All Features	0.518	0.566	0.537	0.551	28	Weak
RFE Selected	0.567	0.606	0.610	0.608	10	Improved
Hypertuned	0.567	0.606	0.610	0.608	10	No change

8.3 Key Observations and Insights

8.3.1 Regression Model Analysis

- **Linear Models Dominate:** Ridge regression achieved the best performance with Test $R^2 = 0.0085$
- **Complex Models Fail:** Decision Tree $R^2 = -1.300$ indicates severe overfitting to noise
- **Hyperparameter Benefits Limited:** Tuning improved Ridge only marginally
- **Negative R^2 Warning:** Most models perform worse than predicting the mean return
- **Low Signal-to-Noise:** RMSE 2.5% represents inherent market noise

8.3.2 Classification Model Analysis

- **Feature Selection Impact:** RFE-selected features improved most models except Decision Tree
- **Hypertuning Creates Bias:** Tuned models achieve high recall (99%) but poor precision
- **Trading Implications:** High recall models generate many false buy signals
- **Model Ranking:** Decision Tree (tuned) > SVM (tuned) > Logistic Regression > KNN
- **Accuracy Limitation:** All models hover around 45-56% accuracy (barely better than random)

8.3.3 Cross-Problem Insights

- **Problem Difficulty:** Both regression and classification struggle with stock prediction
- **Overfitting Tendency:** Complex models consistently overfit to training noise
- **Feature Engineering Critical:** Proper lag implementation prevents data leakage
- **Evaluation Metric Choice:** Standard ML metrics may not reflect trading viability

8.3.4 Model Behavior Patterns

The hyperparameter-tuned classification models exhibit concerning behavior:

```
1 # Tuned Decision Tree Confusion Matrix
2 [[ 2 109] # Only 2 correct "down" predictions
3  [ 1 135]] # Almost perfect "up" predictions
4
5 # This pattern indicates:
6 # - Model predicts "UP" for 99% of cases
7 # - High recall but unreliable for actual trading
8 # - Optimized for metric rather than practical utility
```

Listing 30: High Recall Pattern Analysis

9 Lessons Learned

9.1 Successful Approaches

9.1.1 Regression Models

1. **Linear Model Superiority:** Simple linear models outperformed complex algorithms
2. **Ridge Regularization:** Slight regularization ($\lambda=100.0$) improved generalization
3. **Feature Engineering:** Proper lag implementation prevented data leakage
4. **VIF Analysis:** Successfully reduced multicollinearity from 28 to 10 features

9.1.2 Classification Models

1. **Feature Selection Value:** RFE improved most models except Decision Tree
2. **Balanced Evaluation:** Raw models often provided better precision-recall balance
3. **Model-Specific Behavior:** KNN benefited most from feature selection
4. **Hyperparameter Caution:** Tuning can optimize wrong objectives in financial contexts

9.2 Unsuccessful Approaches

9.2.1 Regression Models

1. **Complex Models:** Tree-based and kernel methods severely overfitted
2. **Lasso Regularization:** Too aggressive, eliminated important signals
3. **Traditional ML Metrics:** R^2 not suitable for evaluating stock prediction models
4. **High-dimensional Features:** Initial 28 features led to overfitting

9.2.2 Classification Models

1. **Hyperparameter Optimization:** Created biased models with unrealistic recall
2. **Standard Accuracy Metrics:** Misleading in imbalanced financial datasets
3. **Feature Selection for Trees:** Decision Trees perform better with all features
4. **Precision Sacrifice:** High recall came at cost of too many false signals

9.3 Critical Insights

9.3.1 Financial Market Reality

1. **Market Efficiency:** Stock prediction inherently difficult due to efficient markets
2. **Signal-to-Noise Ratio:** Extremely low in financial time series data
3. **Non-Stationarity:** Returns more stationary than prices but still challenging
4. **Noise vs Pattern:** Models often capture noise rather than genuine patterns

9.3.2 Methodological Lessons

1. **Evaluation Metrics:** Financial models need domain-specific evaluation criteria
2. **Cross-Validation:** Time series requires careful temporal splitting
3. **Model Interpretability:** Simple models more reliable than black boxes
4. **Practical Viability:** Academic performance doesn't guarantee trading success

9.4 Model Selection Guidelines

Based on the comprehensive analysis, we provide the following recommendations:

9.4.1 For Regression Tasks

- **Primary Choice:** Ridge Regression with moderate regularization
- **Alternative:** Linear Regression if dataset is small
- **Avoid:** Complex models (Decision Trees, SVR) due to overfitting tendency

9.4.2 For Classification Tasks

- **Balanced Performance:** Raw SVM with all features
- **High Recall Needs:** Hypertuned models (with caution about false signals)
- **Avoid:** Over-optimized models that predict single class predominantly

10 Future Improvements

10.1 Advanced Models

- ARIMA for autoregressive patterns
- LSTM for sequential dependencies
- GARCH for volatility modeling

10.2 Feature Enhancement

- Market sentiment indicators
- Macroeconomic variables
- Sector performance metrics
- News sentiment analysis

10.3 Ensemble Approaches

- Voting classifiers
- Stacking regressors
- Blend predictions from different time horizons

10.4 Alternative Targets

- 3-5 day returns (less noisy)
- Volatility prediction
- Risk-adjusted returns
- Regime classification

11 Conclusion

This comprehensive project demonstrates the inherent challenges in stock price prediction using traditional machine learning approaches across both regression and classification paradigms. The extensive analysis reveals fundamental insights about financial market predictability and the limitations of conventional ML methods.

11.1 Performance Summary

11.1.1 Regression Results

While regression models achieved reasonable RMSE values (2.5%), the near-zero R^2 scores (best: 0.0085 for Ridge) highlight that stock returns are largely unpredictable using historical price and volume data alone. The performance hierarchy clearly favored simple linear models over complex alternatives:

- Ridge Regression: Best overall performance with minimal overfitting
- Linear Regression: Comparable performance, more interpretable
- Complex Models: Severe overfitting with negative R^2 values

11.1.2 Classification Results

Classification models showed more varied performance (45-56% accuracy) but revealed concerning patterns in hyperparameter-optimized versions. The key findings include:

- Raw models provided better precision-recall balance
- Tuned models achieved high recall (99%) at the cost of precision
- Feature selection improved most models except Decision Trees
- All accuracies remained close to random guessing (50%)

11.2 Key Methodological Contributions

11.2.1 Technical Achievements

1. **Comprehensive Pipeline:** Implemented complete ML workflow with proper validation
2. **Data Leakage Prevention:** Systematic use of lagged features and proper time series splitting
3. **Multicollinearity Treatment:** VIF analysis reduced features from 28 to 10 effectively
4. **Dual Problem Approach:** Addressed both regression and classification formulations
5. **Hyperparameter Caution:** Demonstrated risks of over-optimization in financial contexts

11.2.2 Evaluation Framework

The project established that traditional ML metrics can be misleading in financial applications:

- R^2 near zero is expected, not indicative of model failure

- High recall without precision is dangerous for trading applications
- Domain-specific metrics (directional accuracy, hit rate) more relevant
- Cross-validation must respect temporal structure

11.3 Practical Implications

11.3.1 For Practitioners

1. **Model Selection:** Prefer simple, interpretable models over complex algorithms
2. **Feature Engineering:** Invest heavily in domain-specific feature creation
3. **Evaluation Metrics:** Use trading-relevant performance measures
4. **Risk Management:** Account for model limitations in trading strategies

11.3.2 For Researchers

1. **Alternative Data:** Historical prices alone insufficient for prediction
2. **Time Series Methods:** ARIMA, GARCH may be more appropriate than standard ML
3. **Ensemble Approaches:** Combine multiple weak predictors carefully
4. **Regime Detection:** Focus on identifying market states rather than point predictions

11.4 Fundamental Insights

The comprehensive analysis reveals several fundamental truths about financial market prediction:

11.4.1 Market Efficiency Validated

Both regression and classification results support the Efficient Market Hypothesis. The consistently poor performance across multiple algorithms and configurations suggests that:

- Historical price patterns do not reliably predict future movements
- Market information is quickly incorporated into prices
- Profitable patterns, if they exist, are rapidly arbitrated away

11.4.2 Methodology Matters More Than Algorithms

The project demonstrates that proper methodology (feature engineering, data leakage prevention, appropriate evaluation) is more critical than algorithm choice. Simple linear models with good methodology outperformed complex algorithms with poor practices.

11.4.3 Domain Knowledge Essential

Financial time series require specialized treatment that differs significantly from standard ML applications. Understanding market microstructure, behavioral finance, and regulatory environments is crucial for meaningful progress.

11.5 Future Research Directions

Based on these findings, we recommend future research focus on:

1. **Alternative Data Sources:** Sentiment analysis, macroeconomic indicators, news events
2. **Time Series Specialization:** LSTM, Transformer architectures, regime-switching models
3. **Risk-Adjusted Metrics:** Sharpe ratio optimization, maximum drawdown constraints
4. **Multi-Asset Approaches:** Portfolio-level prediction rather than individual stocks
5. **Behavioral Factors:** Incorporating investor psychology and market sentiment

11.6 Final Assessment

This project successfully implements a complete machine learning pipeline for stock price prediction but confirms the fundamental difficulty of beating market efficiency with conventional approaches. The research contributes to our understanding of machine learning limitations in financial markets and provides a rigorous foundation for future enhancements.

The results suggest that while machine learning has transformative potential in many domains, financial market prediction remains one of the most challenging applications. Success in this field likely requires combining domain expertise, alternative data sources, sophisticated risk management, and realistic expectations about what is achievable in efficient markets.

Future practitioners should view these results not as evidence that prediction is impossible, but as a call for more sophisticated approaches that respect the unique challenges of financial time series while leveraging the growing availability of alternative data sources and advanced modeling techniques.

12 References

1. Detecting and Analysing Possible Outliers in Global Stock Market Returns, *Taylor & Francis Online*, 2022.
2. Survey of feature selection and extraction techniques for stock market prediction, *Financial Innovation*, 2022.
3. Evaluating Outlier Impact on Time Series Data Analysis, *Magnimind Academy*, 2023.

4. Yahoo Finance API Documentation. Available at: <https://pypi.org/project/yfinance/>